

1 : [Implement basic array operations: insertion, deletion, searching, and traversal.](#)

```
#include <stdio.h>

int main() {

    int arr[100], n, i, pos, value, choice, found;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    do {
        printf("\n--- MENU ---\n");
        printf("1. Traversal\n");
        printf("2. Insertion\n");
        printf("3. Deletion\n");
        printf("4. Searching\n");
        printf("5. Exit\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);
```

```
switch(choice) {
```

```
    case 1: // Traversal
```

```
        printf("Array elements are: ");
```

```
        for(i = 0; i < n; i++) {
```

```
            printf("%d ", arr[i]);
```

```
        }
```

```
        printf("\n");
```

```
        break;
```

```
    case 2: // Insertion
```

```
        printf("Enter position and value: ");
```

```
        scanf("%d %d", &pos, &value);
```

```
        if(pos < 1 || pos > n + 1) {
```

```
            printf("Invalid position!\n");
```

```
        } else {
```

```
            for(i = n; i >= pos; i--) {
```

```
                arr[i] = arr[i - 1];
```

```
            }
```

```
            arr[pos - 1] = value;
```

```
            n++;
```

```
    printf("Element inserted successfully.\n");  
}  
break;
```

case 3: // Deletion

```
    printf("Enter position to delete: ");  
    scanf("%d", &pos);
```

```
    if(pos < 1 || pos > n) {  
        printf("Invalid position!\n");  
    } else {  
        for(i = pos - 1; i < n - 1; i++) {  
            arr[i] = arr[i + 1];  
        }
```

```
        n--;
```

```
        printf("Element deleted successfully.\n");  
    }  
    break;
```

case 4: // Searching

```
    printf("Enter element to search: ");
```

```
scanf("%d", &value);
```

```
found = 0;
```

```
for(i = 0; i < n; i++) {
```

```
    if(arr[i] == value) {
```

```
        printf("Element found at position %d\n", i + 1);
```

```
        found = 1;
```

```
        break;
```

```
    }
```

```
}
```

```
if(found == 0) {
```

```
    printf("Element not found\n");
```

```
}
```

```
break;
```

```
case 5:
```

```
    printf("Exiting...\n");
```

```
    break;
```

```
default:
```

```
    printf("Invalid choice!\n");
```

```

    }

    } while(choice != 5);

    return 0;
}

```

Output :

```

C:\Users\Kusuma B S\Downl... x + v
Enter number of elements: 4
Enter elements:
1 2 3 4

--- MENU ---
1. Traversal
2. Insertion
3. Deletion
4. Searching
5. Exit
Enter your choice: 2
Enter position and value: 1
2
Element inserted successfully.

--- MENU ---
1. Traversal
2. Insertion
3. Deletion
4. Searching
5. Exit
Enter your choice: 5
Exiting...

Process returned 0 (0x0)   execution time : 32.831 s
Press any key to continue.

```

2 : Represent a sparse matrix using triplet form and perform transpose operation.

```
#include <stdio.h>
```

```

int main() {

    int matrix[10][10], triplet[50][3], transpose[50][3];

    int i, j, m, n, k = 1;

```

```
printf("Enter rows and columns: ");  
scanf("%d %d", &m, &n);
```

```
printf("Enter matrix elements:\n");  
for(i = 0; i < m; i++) {  
    for(j = 0; j < n; j++) {  
        scanf("%d", &matrix[i][j]);  
    }  
}
```

```
// Convert to triplet form
```

```
triplet[0][0] = m;  
triplet[0][1] = n;  
triplet[0][2] = 0;
```

```
for(i = 0; i < m; i++) {  
    for(j = 0; j < n; j++) {  
        if(matrix[i][j] != 0) {  
            triplet[k][0] = i;  
            triplet[k][1] = j;  
            triplet[k][2] = matrix[i][j];  
            k++;  
            triplet[0][2]++;  
        }  
    }  
}
```

```
    }  
}
```

```
printf("\nTriplet Representation:\n");  
for(i = 0; i < k; i++) {  
    printf("%d %d %d\n", triplet[i][0], triplet[i][1], triplet[i][2]);  
}
```

```
// Transpose
```

```
transpose[0][0] = triplet[0][1];  
transpose[0][1] = triplet[0][0];  
transpose[0][2] = triplet[0][2];
```

```
k = 1;
```

```
for(i = 0; i < n; i++) {  
    for(j = 1; j <= triplet[0][2]; j++) {  
        if(triplet[j][1] == i) {  
            transpose[k][0] = triplet[j][1];  
            transpose[k][1] = triplet[j][0];  
            transpose[k][2] = triplet[j][2];  
            k++;  
        }  
    }  
}
```

```

    }

    printf("\nTranspose Triplet:\n");

    for(i = 0; i < k; i++) {

        printf("%d %d %d\n", transpose[i][0], transpose[i][1],
transpose[i][2]);

    }

    return 0;

}

```

Output:

```

C:\Users\Kusuma B S\Downl... x + v
Enter rows and columns: 3 3
Enter matrix elements:
1 2 3
4 5 6
7 8 9

Triplet Representation:
3 3 9
0 0 1
0 1 2
0 2 3
1 0 4
1 1 5
1 2 6
2 0 7
2 1 8
2 2 9

Transpose Triplet:
3 3 9
0 0 1
0 1 4
0 2 7
1 0 2
1 1 5
1 2 8
2 0 3
2 1 6
2 2 9

Process returned 0 (0x0)   execution time : 15.339 s
Press any key to continue.

```

3: [Implement stack operations using arrays and linked lists.](#)

```
#include <stdio.h>
```

```
#define MAX 5
```

```
int stack[MAX];
```

```
int top = -1;
```

```
// Push operation
```

```
void push(int value) {
```

```
    if (top == MAX - 1) {
```

```
        printf("Stack Overflow\n");
```

```
    } else {
```

```
        top++;
```

```
        stack[top] = value;
```

```
        printf("%d pushed into stack\n", value);
```

```
    }
```

```
}
```

```
// Pop operation
```

```
void pop() {
```

```
    if (top == -1) {
```

```
        printf("Stack Underflow\n");
```

```
    } else {
```

```
        printf("%d popped from stack\n", stack[top]);
```

```
        top--;
```

```
    }
```

```
}
```

```
// Display operation
```

```
void display() {  
    if (top == -1) {  
        printf("Stack is empty\n");  
    } else {  
        printf("Stack elements are:\n");  
  
        for (int i = top; i >= 0; i--) {  
            printf("%d\n", stack[i]);  
        }  
    }  
}
```

// Main function

```
int main() {
```

```
    push(10);
```

```
    push(20);
```

```
    push(30);
```

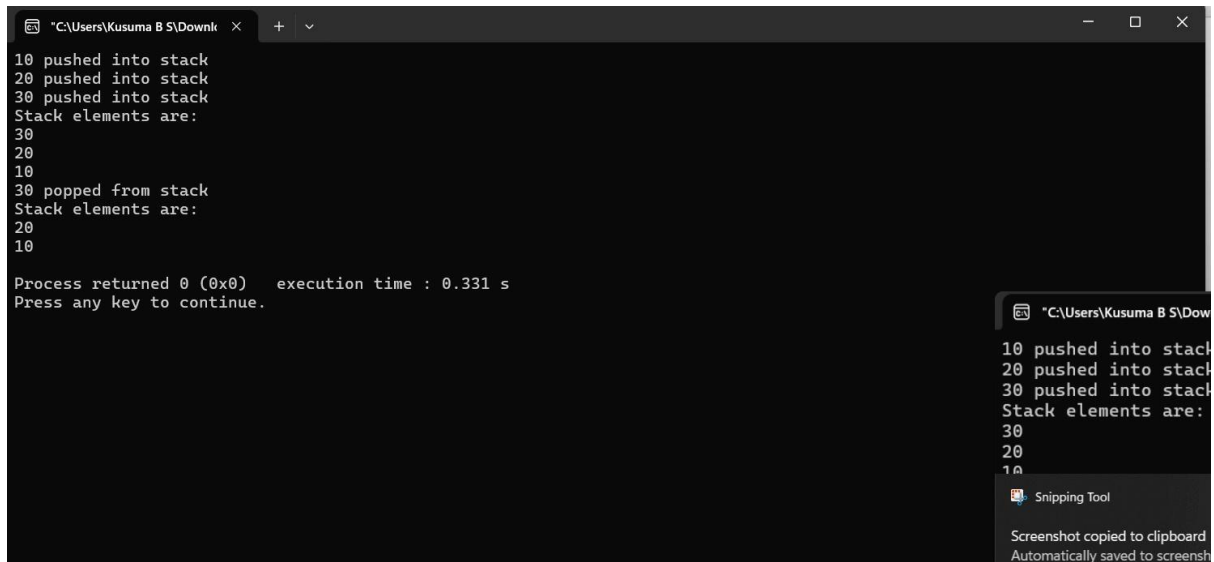
```
    display();
```

```
    pop();
```

```
    display();
```

```
    return 0;
}
```

Output:



```
"C:\Users\Kusuma B S\Downk x + v
10 pushed into stack
20 pushed into stack
30 pushed into stack
Stack elements are:
30
20
10
30 popped from stack
Stack elements are:
20
10
Process returned 0 (0x0)   execution time : 0.331 s
Press any key to continue.
```

4: [Convert an infix expression to postfix using a stack.](#)

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <ctype.h>
```

```
#define MAX 100
```

```
char stack[MAX];
```

```
int top = -1;
```

```
void push(char c) {
```

```
if (top >= MAX - 1) {  
    printf("Stack Overflow\n");  
    return;  
}  
stack[++top] = c;  
}
```

```
char pop() {  
    if (top < 0) {  
        return '\0';  
    }  
    return stack[top--];  
}
```

```
char peek() {  
    if (top < 0) {  
        return '\0';  
    }  
    return stack[top];  
}
```

```
int precedence(char op) {  
    switch (op) {  
        case '+':
```

```

        case '-':
            return 1;
        case '*':
        case '/':
            return 2;
        case '^':
            return 3;
        default:
            return 0;
    }
}

int isOperator(char c) {
    return (c == '+' || c == '-' || c == '*' || c == '/' || c == '^');
}

void infixToPostfix(char *infix, char *postfix) {
    int i = 0, j = 0;
    char c;
    while ((c = infix[i++]) != '\0') {
        // If operand, add to output
        if (isalnum(c)) {
            postfix[j++] = c;
        }
        // If '(', push to stack
        else if (c == '(') {

```

```

    push(c);
}
// If ')', pop until '(' is found
else if (c == ')') {
    while (top >= 0 && peek() != '(') {
        postfix[j++] = pop();
    }
    pop(); // Remove '(' from stack
}
// If operator
else if (isOperator(c)) {
    // Handle right-associativity for '^'
    if (c == '^') {
        while (top >= 0 && precedence(peek()) > precedence(c)) {
            postfix[j++] = pop();
        }
    } else {
        while (top >= 0 && precedence(peek()) >= precedence(c)) {
            postfix[j++] = pop();
        }
    }
    push(c);
}
}

```

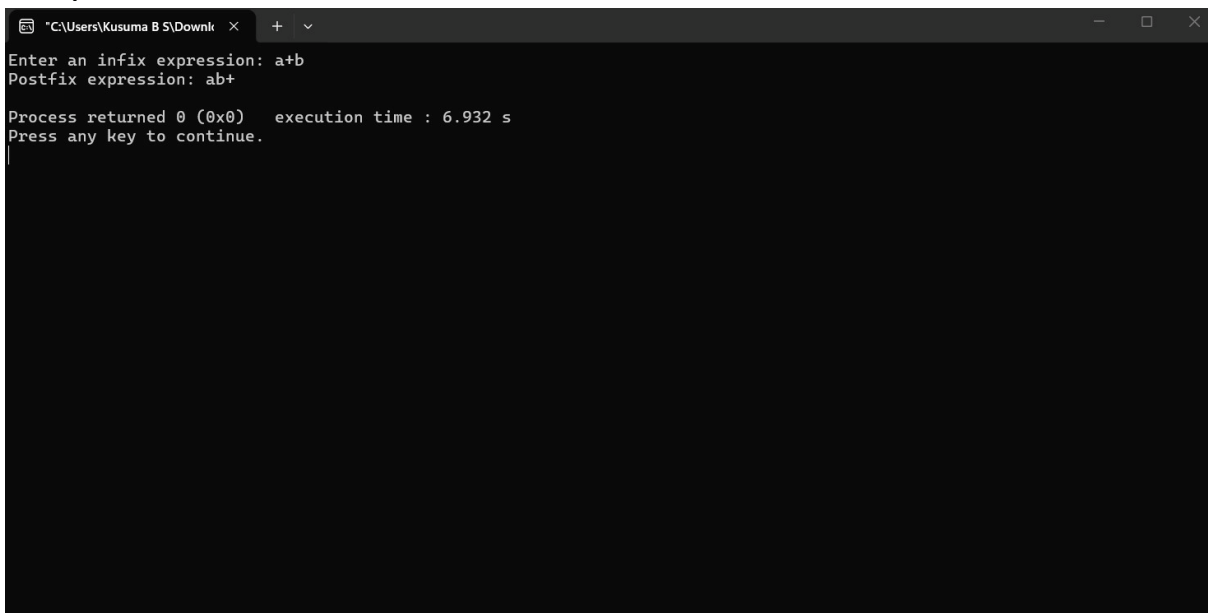
```

// Pop remaining operators
while (top >= 0) {
    postfix[j++] = pop();
}
postfix[j] = '\0';
}

int main() {
    char infix[MAX], postfix[MAX];
    printf("Enter an infix expression: ");
    scanf("%s", infix);
    infixToPostfix(infix, postfix);
    printf("Postfix expression: %s\n", postfix);
    return 0;
}

```

Output:



```

C:\Users\Kusuma B S\Downl...
Enter an infix expression: a+b
Postfix expression: ab+

Process returned 0 (0x0)   execution time : 6.932 s
Press any key to continue.

```

5:a: [Queue operations using arrays](#)

```
#include <stdio.h>
```

```
#define MAX 5
```

```
int queue[MAX];
```

```
int front = -1, rear = -1;
```

```
// Insert element
```

```
void enqueue(int value)
```

```
{
```

```
    if(rear == MAX - 1)
```

```
    {
```

```
        printf("Queue Overflow\n");
```

```
    }
```

```
    else
```

```
    {
```

```
        if(front == -1)
```

```
            front = 0;
```

```
        rear++;
```

```
        queue[rear] = value;
```

```
        printf("%d inserted\n", value);
```

```
    }
```

```
}
```

```
// Delete element
```

```
void dequeue()
```

```
{
```

```
    if(front == -1 || front > rear)
```

```
    {
```

```
        printf("Queue Underflow\n");
```

```
    }
```

```
    else
```

```
    {
```

```
        printf("%d deleted\n", queue[front]);
```

```
        front++;
```

```
    }
```

```
}
```

```
// Display queue
```

```
void display()
```

```
{
```

```
    int i;
```

```
    if(front == -1 || front > rear)
```

```
    {
```

```
        printf("Queue is Empty\n");
```

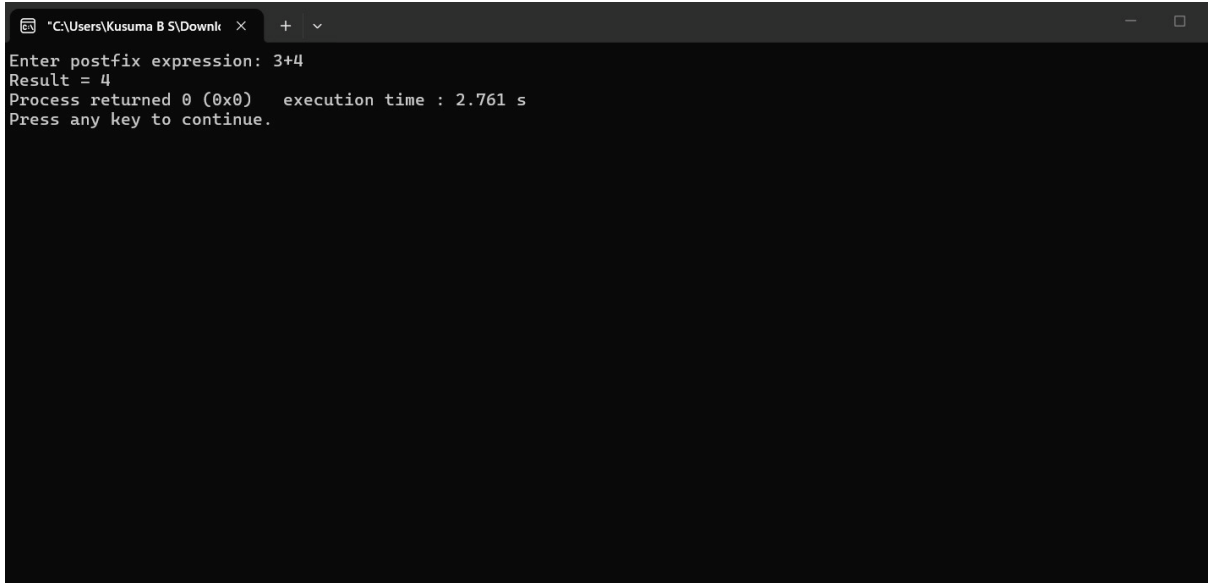
```
}  
else  
{  
    printf("Queue Elements:\n");  
  
    for(i = front; i <= rear; i++)  
    {  
        printf("%d ", queue[i]);  
    }  
}  
  
printf("\n");  
}
```

```
int main()  
{  
    enqueue(10);  
    enqueue(20);  
    enqueue(30);  
  
    display();  
  
    dequeue();  
}
```

```
display();

return 0;
}
```

Output:



```
"C:\Users\Kusuma B S\Downl... X + v
Enter postfix expression: 3+4
Result = 4
Process returned 0 (0x0) execution time : 2.761 s
Press any key to continue.
```

5:b: [Queue operations using Linked List](#)

```
#include <stdio.h>

#include <stdlib.h>

// Node structure
struct Node
{
    int data;
    struct Node *next;
};
```

```
struct Node *front = NULL;
```

```
struct Node *rear = NULL;
```

```
// Insert element
```

```
void enqueue(int value)
```

```
{
```

```
    struct Node *newNode;
```

```
    newNode = (struct Node*)malloc(sizeof(struct Node));
```

```
    newNode->data = value;
```

```
    newNode->next = NULL;
```

```
    if(rear == NULL)
```

```
    {
```

```
        front = rear = newNode;
```

```
    }
```

```
    else
```

```
    {
```

```
        rear->next = newNode;
```

```
        rear = newNode;
```

```
    }
```

```
    printf("%d inserted\n", value);
}

// Delete element
void dequeue()
{
    struct Node *temp;

    if(front == NULL)
    {
        printf("Queue Underflow\n");
    }
    else
    {
        temp = front;

        printf("%d deleted\n", front->data);

        front = front->next;

        if(front == NULL)
            rear = NULL;

        free(temp);
    }
}
```

```
    }  
}  
  
// Display queue  
void display()  
{  
    struct Node *temp;  
  
    if(front == NULL)  
    {  
        printf("Queue is Empty\n");  
    }  
    else  
    {  
        temp = front;  
  
        printf("Queue Elements:\n");  
  
        while(temp != NULL)  
        {  
            printf("%d ", temp->data);  
            temp = temp->next;  
        }  
    }  
}
```

```
    printf("\n");  
}
```

```
int main()  
{  
    enqueue(10);  
    enqueue(20);  
    enqueue(30);  
  
    display();  
  
    dequeue();  
  
    display();  
  
    return 0;  
}
```

Output :

```
"C:\Users\Kusuma B S\Downl... x + v
10 inserted
20 inserted
30 inserted
Queue Elements:
10 20 30
10 deleted
Queue Elements:
20 30

Process returned 0 (0x0)   execution time : 0.421 s
Press any key to continue.
|
```

6:Implement a circular queue using array .

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Node structure
```

```
struct Node
```

```
{
```

```
    int data;
```

```
    struct Node *next;
```

```
};
```

```
struct Node *front = NULL;
```

```
struct Node *rear = NULL;
```

```
// Insert element
void enqueue(int value)
{
    struct Node *newNode;

    newNode = (struct Node*)malloc(sizeof(struct Node));

    newNode->data = value;
    newNode->next = NULL;

    if(rear == NULL)
    {
        front = rear = newNode;
    }
    else
    {
        rear->next = newNode;
        rear = newNode;
    }

    printf("%d inserted\n", value);
}

// Delete element
```

```
void dequeue()
{
    struct Node *temp;

    if(front == NULL)
    {
        printf("Queue Underflow\n");
    }
    else
    {
        temp = front;

        printf("%d deleted\n", front->data);

        front = front->next;

        if(front == NULL)
            rear = NULL;

        free(temp);
    }
}

// Display queue
```

```
void display()
{
    struct Node *temp;

    if(front == NULL)
    {
        printf("Queue is Empty\n");
    }
    else
    {
        temp = front;

        printf("Queue Elements:\n");

        while(temp != NULL)
        {
            printf("%d ", temp->data);
            temp = temp->next;
        }

        printf("\n");
    }
}
```

```
int main()
{
    enqueue(10);
    enqueue(20);
    enqueue(30);

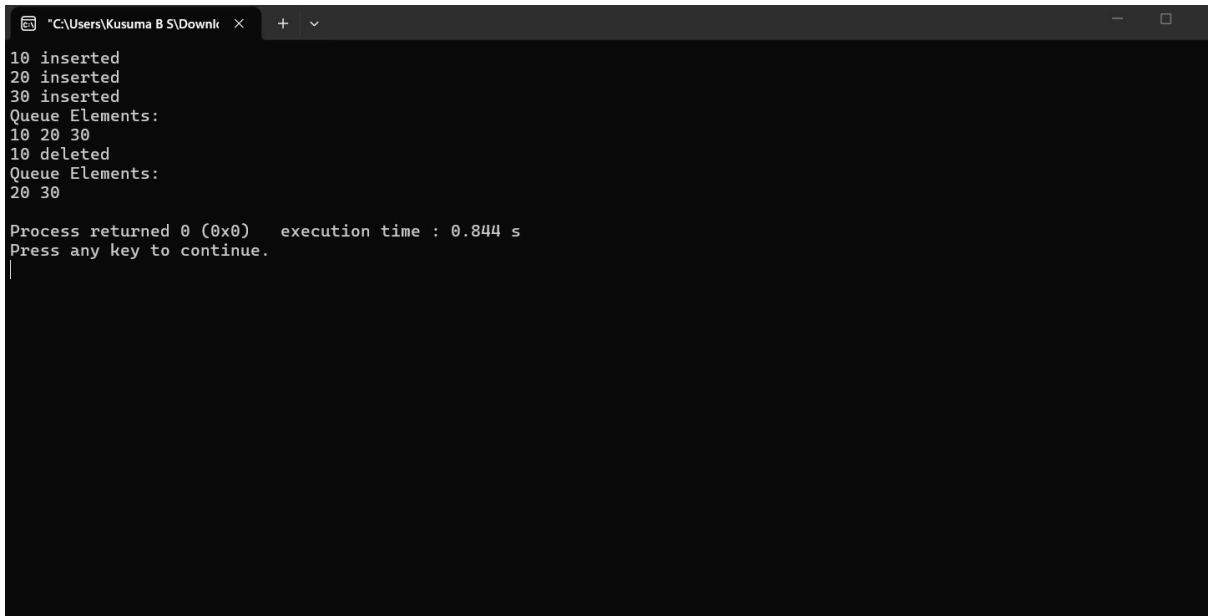
    display();

    dequeue();

    display();

    return 0;
}
```

Output :



```
"C:\Users\Kusuma B S\Downl...  x  +  v
10 inserted
20 inserted
30 inserted
Queue Elements:
10 20 30
10 deleted
Queue Elements:
20 30

Process returned 0 (0x0)   execution time : 0.844 s
Press any key to continue.
|
```

7: Implement a priority queue using arrays or linked lists.

```
#include <stdio.h>
```

```
#define MAX 5
```

```
int pq[MAX];
```

```
int size = 0;
```

```
// Insert element in sorted order
```

```
void enqueue(int value)
```

```
{
```

```
    int i;
```

```
    if(size == MAX)
```

```
    {
```

```
        printf("Priority Queue Overflow\n");
```

```
        return;
```

```
    }
```

```
// Insert based on priority (smaller number = higher priority)
```

```
for(i = size - 1; i >= 0 && pq[i] > value; i--)
```

```
{
```

```
    pq[i + 1] = pq[i];
```

```
}
```

```
    pq[i + 1] = value;
    size++;

    printf("%d inserted\n", value);
}

// Delete highest priority element
void dequeue()
{
    int i;

    if(size == 0)
    {
        printf("Priority Queue Underflow\n");
        return;
    }

    printf("%d deleted\n", pq[0]);

    for(i = 0; i < size - 1; i++)
    {
        pq[i] = pq[i + 1];
    }
}
```

```
    size--;  
}  
  
// Display elements  
void display()  
{  
    int i;  
  
    if(size == 0)  
    {  
        printf("Priority Queue is Empty\n");  
    }  
    else  
    {  
        printf("Priority Queue Elements:\n");  
  
        for(i = 0; i < size; i++)  
        {  
            printf("%d ", pq[i]);  
        }  
    }  
  
    printf("\n");
```

```
}
```

```
int main()
```

```
{
```

```
    enqueue(30);
```

```
    enqueue(10);
```

```
    enqueue(50);
```

```
    enqueue(20);
```

```
    display();
```

```
    dequeue();
```

```
    display();
```

```
    return 0;
```

```
}
```

Output:

```
*C:\Users\Kusuma B S\Downl... x + v
10 inserted
20 inserted
30 inserted
Queue Elements:
10 20 30
10 deleted
Queue Elements:
20 30

Process returned 0 (0x0)   execution time : 0.396 s
Press any key to continue.
```

8: [Implement singly linked list operations: insertion, deletion, and search.](#)

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {
```

```
    int data;
```

```
    struct Node* next;
```

```
};
```

```
struct Node* createNode(int data) {
```

```
    struct Node* newNode = (struct Node*)malloc(sizeof(struct
Node));
```

```
    newNode->data = data;
```

```
    newNode->next = NULL;
```

```
    return newNode;
```

```
}
```

```
void insertAtBeginning(struct Node** head, int data) {  
    struct Node* newNode = createNode(data);  
    newNode->next = *head;  
    *head = newNode;  
}
```

```
void insertAtEnd(struct Node** head, int data) {  
    struct Node* newNode = createNode(data);  
    if (*head == NULL) {  
        *head = newNode;  
        return;  
    }  
    struct Node* temp = *head;  
    while (temp->next != NULL) {  
        temp = temp->next;  
    }  
    temp->next = newNode;  
}
```

```
void insertAtPosition(struct Node** head, int data, int position) {  
    struct Node* newNode = createNode(data);  
    if (position == 0) {
```

```

    newNode->next = *head;
    *head = newNode;
    return;
}
struct Node* temp = *head;
for (int i = 0; temp != NULL && i < position - 1; i++) {
    temp = temp->next;
}
if (temp == NULL) {
    printf("Position out of range\n");
    return;
}
newNode->next = temp->next;
temp->next = newNode;
}

```

```

void deleteFromBeginning(struct Node** head) {
    if (*head == NULL) return;
    struct Node* temp = *head;
    *head = (*head)->next;
    free(temp);
}

```

```

void deleteFromEnd(struct Node** head) {

```

```

if (*head == NULL) return;
if ((*head)->next == NULL) {
    free(*head);
    *head = NULL;
    return;
}
struct Node* temp = *head;
while (temp->next->next != NULL) {
    temp = temp->next;
}
free(temp->next);
temp->next = NULL;
}

void deleteFromPosition(struct Node** head, int position) {
    if (*head == NULL) return;
    struct Node* temp = *head;
    if (position == 0) {
        *head = temp->next;
        free(temp);
        return;
    }
    for (int i = 0; temp != NULL && i < position - 1; i++) {
        temp = temp->next;
    }
}

```

```
}  
if (temp == NULL || temp->next == NULL) {  
    printf("Position out of range\n");  
    return;  
}  
struct Node* next = temp->next->next;  
free(temp->next);  
temp->next = next;  
}
```

```
void traverse(struct Node* head) {  
    struct Node* temp = head;  
    while (temp != NULL) {  
        printf("%d -> ", temp->data);  
        temp = temp->next;  
    }  
    printf("NULL\n");  
}
```

```
int search(struct Node* head, int target) {  
    struct Node* temp = head;  
    while (temp != NULL) {  
        if (temp->data == target) return 1;  
        temp = temp->next;  
    }
```

```
}  
return 0;  
}
```

```
void reverse(struct Node** head) {  
    struct Node* prev = NULL;  
    struct Node* current = *head;  
    struct Node* next = NULL;  
    while (current != NULL) {  
        next = current->next;  
        current->next = prev;  
        prev = current;  
        current = next;  
    }  
    *head = prev;  
}
```

```
void sort(struct Node** head) {  
    if (*head == NULL || (*head)->next == NULL) return;  
    struct Node* temp = *head;  
    while (temp != NULL) {  
        struct Node* next = temp->next;  
        while (next != NULL) {  
            if (temp->data > next->data) {
```

```
        int data = temp->data;
        temp->data = next->data;
        next->data = data;
    }
    next = next->next;
}
temp = temp->next;
}
}
```

```
int length(struct Node* head) {
    int count = 0;
    struct Node* temp = head;
    while (temp != NULL) {
        count++;
        temp = temp->next;
    }
    return count;
}
```

```
int main() {
    struct Node* head = NULL;

    insertAtEnd(&head, 10);
```

```
insertAtEnd(&head, 20);
```

```
insertAtEnd(&head, 30);
```

```
traverse(head);
```

```
insertAtBeginning(&head, 5);
```

```
traverse(head);
```

```
insertAtPosition(&head, 25, 2);
```

```
traverse(head);
```

```
deleteFromBeginning(&head);
```

```
traverse(head);
```

```
deleteFromEnd(&head);
```

```
traverse(head);
```

```
deleteFromPosition(&head, 1);
```

```
traverse(head);
```

```
printf("List contains 20: %s\n", search(head, 20) ? "Yes" : "No");
```

```
printf("List contains 40: %s\n", search(head, 40) ? "Yes" : "No");
```

```
reverse(&head);
```

```
traverse(head);
```

```

insertAtEnd(&head, 15);
insertAtEnd(&head, 5);
traverse(head);

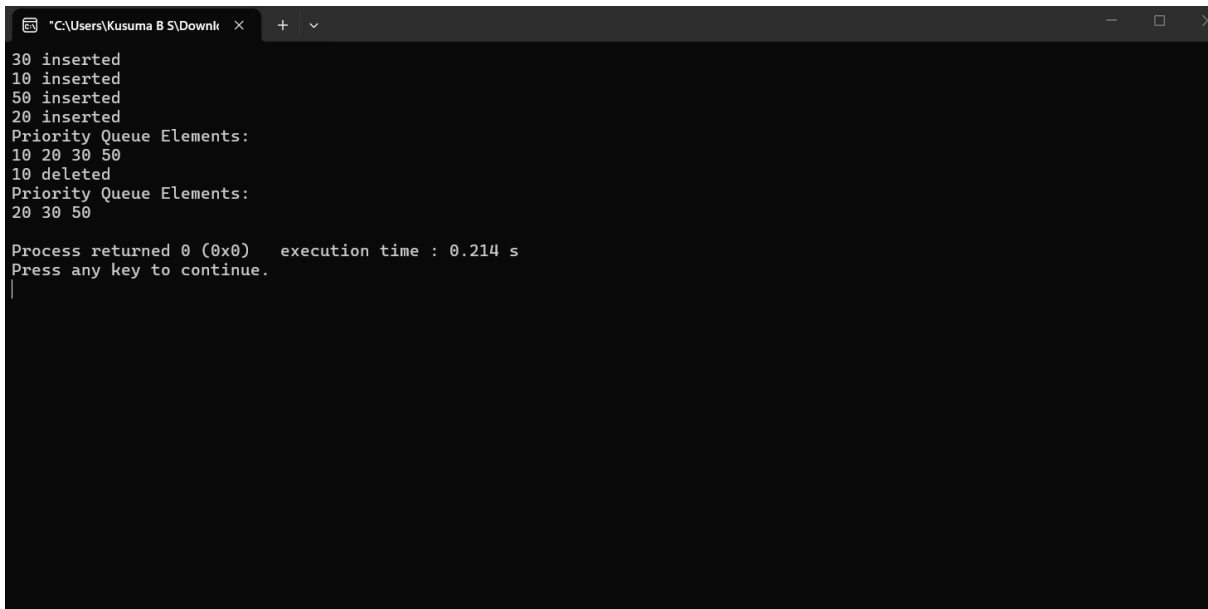
sort(&head);
traverse(head);

printf("Length of the list: %d\n", length(head));

return 0;
}

```

Output:



```

C:\Users\Kusuma B S\Downl... x + v
30 inserted
10 inserted
50 inserted
20 inserted
Priority Queue Elements:
10 20 30 50
10 deleted
Priority Queue Elements:
20 30 50
Process returned 0 (0x0)   execution time : 0.214 s
Press any key to continue.

```

9: [Implement binary tree creation and traversal \(preorder, inorder, postorder\). #include <stdio.h>](#)

```
#include <stdio.h>
#include <stdlib.h>

// Structure for tree node
struct Node
{
    int data;
    struct Node *left;
    struct Node *right;
};

// Create new node
struct Node* createNode(int value)
{
    struct Node *newNode;

    newNode = (struct Node*)malloc(sizeof(struct Node));

    newNode->data = value;
    newNode->left = NULL;
    newNode->right = NULL;

    return newNode;
}
```

// Preorder Traversal

void preorder(struct Node *root)

```
{  
    if(root != NULL)  
    {  
        printf("%d ", root->data);  
        preorder(root->left);  
        preorder(root->right);  
    }  
}
```

// Inorder Traversal

void inorder(struct Node *root)

```
{  
    if(root != NULL)  
    {  
        inorder(root->left);  
        printf("%d ", root->data);  
        inorder(root->right);  
    }  
}
```

// Postorder Traversal

```
void postorder(struct Node *root)
{
    if(root != NULL)
    {
        postorder(root->left);
        postorder(root->right);
        printf("%d ", root->data);
    }
}
```

```
int main()
{
    // Creating Binary Tree
    struct Node *root = createNode(1);

    root->left = createNode(2);
    root->right = createNode(3);

    root->left->left = createNode(4);
    root->left->right = createNode(5);

    // Traversals
    printf("Preorder Traversal:\n");
    preorder(root);
}
```

```

printf("\n\nInorder Traversal:\n");

inorder(root);


printf("\n\nPostorder Traversal:\n");

postorder(root);


return 0;

}

```

Output:

```

10 -> 20 -> 30 -> NULL
5 -> 10 -> 20 -> 30 -> NULL
5 -> 10 -> 25 -> 20 -> 30 -> NULL
10 -> 25 -> 20 -> 30 -> NULL
10 -> 25 -> 20 -> NULL
10 -> 20 -> NULL
List contains 20: Yes
List contains 40: No
20 -> 10 -> NULL
20 -> 10 -> 15 -> 5 -> NULL
5 -> 10 -> 15 -> 20 -> NULL
Length of the list: 4

Process returned 0 (0x0)   execution time : 0.379 s
Press any key to continue.

```

10: [Evaluate a postfix expression using a stack.](#)

```
#include <stdio.h>
```

```
#include <ctype.h>
```

```
#define MAX 100
```

```
int stack[MAX];
```

```
int top = -1;
```

```
// Push operationA
```

```
void push(int value)
```

```
{
```

```
    top++;
```

```
    stack[top] = value;
```

```
}
```

```
// Pop operation
```

```
int pop()
```

```
{
```

```
    return stack[top--];
```

```
}
```

```
int main()
```

```
{
```

```
    char postfix[MAX];
```

```
    int i, op1, op2, result;
```

```
    printf("Enter postfix expression: ");
```

```
    scanf("%s", postfix);
```

```
for(i = 0; postfix[i] != '\0'; i++)
{
    // If character is operand
    if(isdigit(postfix[i]))
    {
        push(postfix[i] - '0');
    }
    // If character is operator
    else
    {
        op2 = pop();
        op1 = pop();

        switch(postfix[i])
        {
            case '+':
                result = op1 + op2;
                break;

            case '-':
                result = op1 - op2;
                break;

            case '*':
```

```
    result = op1 * op2;  
    break;
```

```
case '/':  
    result = op1 / op2;  
    break;
```

```
case '%':  
    result = op1 % op2;  
    break;
```

```
default:  
    printf("Invalid operator");  
    return 1;
```

```
}
```

```
    push(result);
```

```
}
```

```
}
```

```
printf("Result = %d", pop());
```

```
return 0;
```

```
}
```

Output:

```
"C:\Users\Kusuma B S\Downl... x + v
Preorder Traversal:
1 2 4 5 3
Inorder Traversal:
4 2 5 1 3
Postorder Traversal:
4 5 2 3 1
Process returned 0 (0x0)   execution time : 0.348 s
Press any key to continue.
```